

DeliverEats Git Workflow

Branch Structure

Core branches:

- main → stable, production/demo-ready code
- develop → integration branch

Team branches:

- feature/kareem
- feature/sarhan
- feature/hanaa
- feature/basmala

Workflow path:

feature/member → develop → main

Team Responsibilities

Kareem (Project Maintainer)

Responsibilities:

- Core architecture owner
- Final integration owner
- Main branch maintainer
- Conflict resolution
- Final release verification

Can push to:

- feature/kareem
 - develop
 - main
-

Sarhan

Can push to:

- feature/sarhan

Can merge into:

- develop

Cannot push to:

- main
-

Hanaa

Can push to:

- feature/hanaa

Can merge into:

- develop

Cannot push to:

- main
-

Basmala

Can push to:

- feature/basmala

Can merge into:

- develop

Cannot push to:

- main
-

Daily Workflow

Step 1 — Pull latest develop

Before starting work:

```
git checkout develop
git pull origin develop
```

Step 2 — Update your branch

Merge latest develop into your branch:

```
git checkout feature/your-name
git merge develop
```

Step 3 — Work on your module

Examples:

- Kareem → Orders, Auth, Payments
 - Sarhan → Dispatch, Redis, GPS
 - Hanaa → Restaurants, Menu, Reviews
 - Basmala → Rider Portal, Notifications, Testing
-

Step 4 — Commit changes

Use structured commits:

```
git add .
git commit -m "feat: implement order creation"
```

Commit types:

- feat
 - fix
 - refactor
 - test
 - docs
-

Step 5 — Push feature branch

```
git push origin feature/your-name
```

Step 6 — Merge into develop

After review/testing:

```
feature/member → develop
```

Purpose: Integrate team work.

Integration Workflow

Frequency: Every 2–3 days

On develop branch:

Test:

- API routes
- Authentication
- Database migrations
- Order flow
- Dispatch system
- Real-time events
- Payments

Purpose: Catch integration issues early.

Main Branch Workflow

Only Kareem merges develop into main.

When?

1. End of milestone

Examples:

- Authentication completed
 - Orders completed
 - Restaurant system completed
-

2. Before demo/presentation

Always merge stable version to main.

3. Weekly stable release

If develop is tested and stable:

```
develop → main
```

Optional tagging:

```
git tag v0.1
```

Branch Rules

main branch

Rules:

- Never push directly (except maintainer)
 - Must be stable
 - Must be tested
 - Must be demo-ready
-

develop branch

Rules:

- Integration only
- Accept feature merges

- Test before main merge
-

feature branches

Rules:

- Work only in your branch
 - Keep commits small
 - Pull develop frequently
-

Conflict Prevention Rules

1. Never edit another member's module without communication.
 2. Pull develop before coding.
 3. Merge develop into your feature branch daily.
 4. Use agreed naming conventions.
 5. Use shared API response format.
 6. Keep one owner per module.
 7. Keep main protected.
-

Weekly Cycle

Sunday–Thursday:

- Develop features
- Push feature branches
- Merge into develop

Friday:

- Full integration testing
- Bug fixing

Saturday:

- Stable merge develop → main
-

Golden Rule

main = stable

develop = shared testing

feature branches = active development